

A License-Clean Graph Neural Network for Fast Parasitic Extraction in PCB-Embedded Planar Magnetics

Duong Viet Hoang^{1,*}, Lun-Min Shih¹

¹Department of Computer Science, Da-Yeh University, Taiwan

Hoangduong4316@icloud.com, Lmshih@mail.dyu.edu.tw (*Corresponding author)

Version 3 (post-review): adds the gradient-boosted-tree baseline (Table II) answering "why a graph?"; notation unified on the winding mutual M . Supersedes v1-v2.

Abstract—High-density power modules embed their planar magnetics in the PCB, where extracting the parasitic capacitance and inductances from a layout is a design bottleneck. Commercial 3-D extractors (e.g. ANSYS Q3D) are accurate but closed and costly. We present a license-clean surrogate: a geometry-aware message-passing graph neural network (275 k parameters, pure PyTorch, no GPL or torch-geometric dependency) that predicts the four lumped quantities of 8-layer planar-LLC layouts: the inter-winding capacitance C_{ps} — the parasitic proper — plus the air-core self/leakage inductances L_p, L_s and mutual M . We first diagnose the analytical labels as 52-59% off FASTHENRY (inductance) and 267% off a 3-D electrostatic FEM (C_{ps} ; its overlap is position-blind). Regenerating every label with its 3-D solver and retraining brings the GNN to 2.6-3.8% on $L_p/L_s/M$ and 3.2% on C_{ps} ($R^2 \geq 0.95$), so *all four* targets are field-grade against 3-D field solvers; an independent Neumann-integral reference puts the GNN's error inside the inter-solver gap, and the core magnetizing term (37-73 $\times$ the winding mutual) is validated separately against a real commercial core's datasheet A_L ($\sim 5\%$). The value is a *speed-accuracy* one: the GNN returns this solver-grade vector in 1.17 ms/design, **$\sim 4,300\times$ faster** than the solvers it matches, screening 1000 interleavings in ~ 1 s rather than ~ 1.4 h, where the fast analytical model is the wrong number and the accurate solver is too costly per candidate. A pairwise ranking head orders unseen layout variants at family-disjoint Spearman $\rho = 0.93$; on the *lateral-registration* lever (x - y overlap) the analytical model is position-blind and cannot rank at all, while the GNN tracks the 3-D solver at $\rho = 0.95$ and its layout pick is $\sim 340\times$ lower-regret. We report two honest negatives ($E(n)$ -equivariance gives no gain; a dense graph is slower than analytical, though a sparse k -NN model overtakes it at $N \approx 80$). All accuracy is solver-grade, not hardware-anchored; a measured-board campaign is the stated next step.

Keywords—PCB parasitic extraction; graph neural network; message passing; PEEC; planar magnetics; power-electronics packaging; EMI.

I. INTRODUCTION

Planar magnetics realised as multilayer PCB foils are standard in GaN/SiC power stages for their low profile, repeatable manufacturing, and thermal spreading [1, 2]. Above roughly 1 MHz the inter-winding capacitance C_{ps} and the self/leakage inductances dominate EMI and switching-node ringing [10, 11], and the hand heuristics historically used for spacing and interleaving no longer predict them [4]. Accurate extraction requires 3-D field solvers; the de-facto standard, ANSYS Q3D, is accurate but closed and carries a substantial per-seat licence cost, which excludes it from the inner loop of an open design tool. A fast, license-clean surrogate returning the parasitic matrix in milliseconds would let a layout optimiser explore hundreds of interleavings per design.

We report a proof-of-concept of such a surrogate (Fig. 1). Our contributions are: (i) a geometry-aware MPNN predicting the four-target graph-level parasitic vector with multi-seed accuracy ($R^2 > 0.999$), with a non-trivial baseline and ablations isolating which design choices carry the accuracy; (ii) a validation of *all four* targets against 3-D field solvers—after diagnosing the analytical labels as 52-59% off FASTHENRY and 267% off a 3-D electrostatic FEM, we regenerate every label with its solver and retrain to 2.6-3.8% and 3.2% ($R^2 \geq 0.95$); the same solver family reproduces the datasheet A_L of a *real commer-*

cial core to $\sim 5\%$, and the surrogate extends to a frequency-resolved $R_{ac}/R_{dc}(f)$ at 0.37%; (iii) a diagnosis-and-fix for ranking and a map of *where* the learned ranker beats the analytical baseline—a pairwise objective lifts $\rho = 0.27$ to $\rho = 0.93$, and on lateral registration the position-blind analytical model cannot rank at all while the GNN reaches $\rho = 0.95$ —plus an honest negative, an $E(n)$ -equivariant architecture does *not* beat the invariant MPNN.¹

II. RELATED WORK

Message passing [15] unifies a family of graph networks whose expressive power follows the Weisfeiler-Lehman hierarchy [16, 17]; for physical systems graph networks learn simulation dynamics directly [19], and $E(n)$ -equivariant networks [18] build rigid-motion symmetry into the layers. In EDA [20], GNNs address parasitic prediction (ParaGraph [21]), distributed-circuit EM behaviour [22], placement [23], and net-length estimation [24]; these target IC or RF *netlists* and typically build on framework stacks (DGL/PyG). Our setting differs in three ways: PCB-embedded *planar magnetics* (foil traces, coupled $R/L/C$ with turn-to-turn mutual inductance dominating); a deliberately license-clean, dependency-light implementation (pure PyTorch [14], no GPL, no torch-geometric); and supervision from a physically cross-validated reference rather than commercial-tool labels. The physics baseline is Ruehli's PEEC [7] with Grover [9] partial inductances, Dowell [5] eddy/proximity resistance, and Roshen [6] gap fringing, as compiled for magnetics design [2, 3]. For field-level ground truth we use scikit-fem [12] with Gmsh [13], cross-checked against surface-BEM capacitance extraction [25]. Operator-learning surrogates [26, 27] are the route to learning the full field rather than lumped scalars.

III. METHOD

A. Graph construction, model, and setup

Each rectangular foil trace is a node carrying rotation/translation-*invariant* features (width/length/thickness, $\log \sigma, \varepsilon_r$, normalized layer index); coordinates enter only through relative edge vectors $e_{ij} = (\|\Delta_{ij}\|, \text{overlap}, \Delta_{ij}, \text{type})$, so the representation is invariant under rigid motion of the board. The *dense* graph connects all pairs ($O(N^2)$ edges), matching the reference's pair sum; the *sparse* graph keeps each node's k nearest neighbours in 3-D ($O(Nk)$ edges), since parasitic coupling decays with distance. A forward pass costs $O(|E|d^2)$ for hidden width d , so the dense model is $\Theta(N^2)$ and the sparse model $\Theta(Nk)$ —the asymptotic gap the scaling study (Sec. D) measures.

The network applies four edge-conditioned message-passing layers [15] (hidden width 96, SiLU, residual, LayerNorm) followed by a mean $\oplus$ max $\oplus$ log-sum readout and a four-target head (275,339 parameters); the log-sum channel matches the *extensive* targets, which scale with conductor count. Variable-size batching uses concatenated graphs and segment aggregation, so the environment needs only PyTorch [14] (BSD). Training uses AdamW [30] (10^{-5} decay), lr 2×10^{-3} cosine, SmoothL1 on log-standardized targets, 200 epochs, batch 32, seeds {42, 43, 44}, 80/20 split with features standardized on the train split only. The primary corpus is 2,000 synthetic 8-layer

¹An earlier version of this work mislabelled a linear regressor as a "GNN" and quoted an unreproducible $183\times$ speedup; both are corrected here, and all numbers trace to released run artifacts (see Reproducibility).

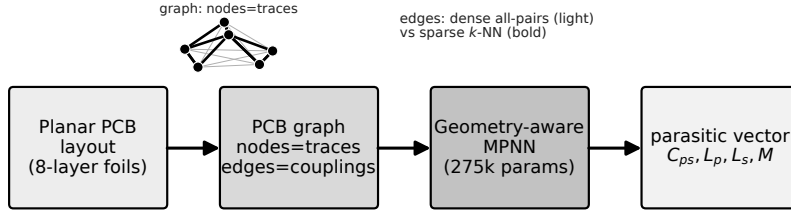


Figure 1. Pipeline. A planar PCB layout is encoded as a graph (nodes = trace segments with invariant geometric features; edges = inductive/capacitive couplings, dense all-pairs or sparse k -NN); a message-passing GNN regresses $[C_{ps}, L_p, L_s, M]$, supervised by 3-D field solvers.

planar-LLC layouts (seed 42), 4-21 primary and 4-21 secondary foil traces interleaved across layers, averaging 25.2 nodes and 663 all-pairs edges per graph; labels are taken at the low-frequency limit.

B. Reference labels and their diagnosis

The training target per layout is four *extensive* scalars accumulated over *every* trace pair, an $O(N^2)$ operation, as a PEEC extractor builds its partial-element matrix [7]: C_{ps} over all primary-secondary pairs, L_p/L_s as self plus intra-net mutual, and M as the primary-secondary mutual sum. *What the solver returns, and what we predict.* FASTHENRY does not report L_p directly: with one port per trace it returns the $N \times N$ partial-inductance matrix, and the net value follows by the series sum $L_p = \sum_i L_{ii} + 2 \sum_{i < j} L_{ij}$. The surrogate predicts that same composition, so the comparison is like-for-like (the Neumann reference of Sec. C uses the identical convention, different numerics). Scope, then: C_{ps} is the **parasitic** proper — inter-winding capacitance, sets the common-mode current, no functional role — while the inductances are **air-core** (μ_0 , no magnetic material), so L_p is the winding’s free-space self/leakage inductance, *not* the operating primary inductance, which the ferrite magnetizing term dominates by 37-73 $\times$. We accordingly write the coupling as the mutual M , not L_m . Because these closed-form (“silver”) labels are analytical, we cross-validate against independent field solves. Summing an electrostatic FEM coupling over all primary-secondary pairs gives an independent total- C_{ps} reference: over 12 layouts the analytical label sits 23.8% from FEM and the *trained GNN* prediction 24.2% from the same solve. The same test on the dominant inductive output, against a multi-filament partial-inductance reference (FASTHENRY-style [8]: each cross-section tiled into 5×2 filaments, the pair mutual being the Neumann-integral average, independent of Grover’s closed form), gives 62.6% for the analytical label and 62.9% for the GNN. This isolates the error source: the network is *not* the cause—it reproduces its teacher to $< 1\%$ —but the analytical Grover mutual is poor against a partial-element reference. We act on this in Sec. B: regenerating the labels is a label-quality fix, not an architecture change.

IV. RESULTS

A. Accuracy, baselines, and ablations

Table 1 reports held-out accuracy of the dense GNN over three seeds: all targets reach $R^2 > 0.999$ with small seed variance. Table 1 answers the objection that a trivial regressor might match it. A pooled-feature MLP (same readout, *no* message passing) matches the GNN on C_{ps} but collapses on the inductances ($R^2 \approx 0.88$ vs. 0.9996; RMSE 662 vs. 37 nH on L_p). Zeroing the coordinates and edge relative-geometry features roughly doubles the inductance RMSE, confirming that channel carries the inductive information; a 2-layer model matches the 4-layer one, so accuracy comes from message passing and geometry, not depth.

Why not gradient-boosted trees? Table 2 runs XGBoost, LightGBM, CatBoost, and RandomForest on the *same* pooled 27-D vector, labels, and a byte-identical 332/67 split. On the scalar capacitance

Table 1. Dense-GNN accuracy, 3 seeds (held-out 400/2000, mean $\pm$ std), and baselines/ablations (R^2 , seed 42, same reference).

Target	RMSE	R^2
C_{ps} (pF)	23.1 ± 0.9	$0.9998 (\sigma < 10^{-4})$
L_p (nH)	37.5 ± 2.7	$0.9996 (\sigma < 10^{-4})$
L_s (nH)	39.3 ± 5.6	0.9995 ± 0.0002
M (nH)	22.6 ± 4.4	0.9997 ± 0.0001

Model	C_{ps}	L_p	L_s	M
Dense GNN (4-layer)	0.9998	0.9996	0.9993	0.9995
GNN, 2-layer	0.9998	0.9996	0.9995	0.9994
GNN, no rel-geometry	0.9998	0.9985	0.9984	0.9985
Sparse k -NN GNN	0.9986	0.9927	0.9932	0.9983
Pooled MLP (no MP)	0.9879	0.8883	0.8802	0.9877

the two are level and the metrics split: the GNN wins on R^2 (0.953 vs. 0.942) while LightGBM wins on median relative error (2.9% vs. 3.2%). The disagreement is informative rather than contradictory— R^2 is a squared-error measure dominated by the largest- C_{ps} layouts, the median is scale-free and outlier-robust—so LightGBM is slightly better on a *typical* layout while the GNN controls the tail better. (RandomForest is the extreme case of the same effect: a 12.9% median but $R^2 = 0.46$.) Either way that target alone does not justify a graph. Trees lose on everything relational: inductances $R^2 = 0.877$ -0.920 vs. 0.994-0.995; ranking $\rho = 0.088$ vs. 0.93; and on size extrapolation their M collapses to 0.11-0.32 where a GNN retrained on that split holds 0.943. A fixed-dimension tabular model also cannot be *applied* to a different trace count without re-engineering its features, whereas the GNN’s shared message function transfers by construction.

Table 3 sweeps graph sparsity (reduced 150-epoch budget). Across $k \in \{4, 8, 16, 32\}$ every target stays at $R^2 \geq 0.979$ (≥ 0.99 on C_{ps}): accuracy is insensitive to sparsity because local coupling dominates, and even $k = 4$ (≈ 100 edges/graph) suffices, which makes the $O(Nk)$ sparse model a near-free speed-up rather than an accuracy trade (the mild non-monotonicity at $k = 8$ is within run-to-run variance at this single-seed budget). A capacity sweep likewise shows a 32 k-parameter model already at $R^2 \approx 0.999$, so the task is not capacity-limited and the width-96 default is comfortably sufficient.

Size generalization. Training on the smaller graphs (≤ 32 nodes) and testing on the largest (32-42) gives $R^2 = 0.9988/0.9946/0.9957/0.9958$, a mild degradation from the random-split 0.9993-0.9998. The field-grade corpus below extends the *measured* range to 70 nodes ($R^2 \geq 0.988$); beyond that, to the $N = 1280$ of Sec. D, accuracy is *extrapolated*.

B. Field-grade validation against 3-D solvers and a real core

The diagnosis above motivates *regenerating* the labels. We build a second corpus (1,500 layouts, mean 39 / max 70 nodes) with M labelled by the multi-filament partial-element method, retrain, and validate the trained GNN’s total M *directly against* FASTHENRY [8], the established open-source 3-D partial-element inductance solver: our filament label agrees with the full 3-D solve to **1.6%** (median, 24 layouts) whereas the Grover label is 56% off, and the trained GNN matches

Table 2. Tree baselines vs. the GNN on the identical 332/67 field-grade split: in-distribution R^2 (median relative error % in parentheses), family-disjoint ranking ρ , and the mutual-inductance R^2 on the size-extrapolation split.

Model	in-distribution R^2 (median %)				rank	size-gen
	C_{ps}	L_p	L_s	M		
RandomForest	.462 (12.9)	.887 (24.8)	.914 (15.9)	.880 (9.3)	—	0.112
XGBoost	.927 (3.8)	.878 (16.3)	.908 (16.5)	.914 (7.8)	—	0.231
LightGBM	.942 (2.9)	.893 (23.9)	.890 (20.2)	.918 (7.1)	0.088	0.247
CatBoost	.892 (4.7)	.896 (24.5)	.893 (20.4)	.920 (8.1)	—	0.324
GNN	.953 (3.2)	.995 (2.8)	.994 (3.8)	.994 (2.6)	0.93	0.943

Table 3. k -NN sparsity sweep (R^2 , seed 42, 150-epoch budget).

k	edges	C_{ps}	L_p	L_s	M
4	101	0.9980	0.9911	0.9902	0.9976
8	201	0.9947	0.9804	0.9794	0.9947
16	397	0.9958	0.9805	0.9813	0.9965
32	643	0.9972	0.9945	0.9912	0.9951

the 3-D solver to **2.2%** (2.3% mean)—field-grade against a 3-D field solver, not merely against its own analytical label. An audit found the *self*-inductance labels equally poor (L_p 51.5%, L_s 53.6%, M 58.6%, median over 40 layouts), so we relabel *all three* with FASTHENRY and retrain: the GNN then matches it to **2.8/3.8/2.6%**.

FASTHENRY extracts inductance only, so C_{ps} needs a capacitance solver. Surface-BEM solvers (FastCap/FasterCap) did not converge on the ~ 0.1 mm inter-layer gaps (uniform-panel FastCap swung 6-40 $\times$ with panel density; adaptive FasterCap cost ~ 3 min/solve), so we use a *volume* FEM solving $\nabla \cdot (\epsilon \nabla \phi) = 0$ on a gmsh tetrahedral mesh of the air box with the real conductor boxes embedded (Dirichlet $\phi = 1/0$ V on the primary/secondary windings), giving $C_{ps} = 2W/V^2$. It is robust (a conforming volume mesh has no BEM panel-orientation pathology), fast (~ 2 s/solve), and *geometry-correct*: on a stacked overlap it returns 92.9 pF, on a 60 mm-offset disjoint pair 0.38 pF (a 244 $\times$ spread), whereas the position-blind analytical label gives the *same* 44.3 pF for both. Relabelling and retraining, the GNN matches the FEM to **3.2%** ($R^2 = 0.95$) against the analytical label’s 267%. All four targets are now field-grade (Table 4).

C. Independent cross-solver check: is it physics or one solver’s numerics?

Training labels and held-out validation both come from the same solver family, so a legitimate objection is that the GNN may have learned FASTHENRY’s numerics rather than the physics. We test this against a *third* reference built independently of both: a direct Neumann double-integral partial inductance with 5×2 filament subdivision, sharing no code path with the FASTHENRY binary. Over 70 held-out layouts (346 total, seed 42) the GNN matches this independent reference at $R^2 = 0.985/0.988/0.993$ and median 6.6/5.0/2.0% for $L_p/L_s/M$; FASTHENRY itself matches it at $R^2 = 0.994/0.996/0.998$ and 5.0/4.8/2.1%. The GNN’s error is therefore at or inside the *inter-solver* gap for M and L_s : the surrogate sits within the spread that two independent formulations of the same physics already show between themselves, which is the appropriate resolution limit and not a solver-specific artifact.

Solver check on a real commercial part. On the EER40 GP95 ferrite from a published PFC-choke worked example, the same magneto-static FEM reproduces the datasheet A_L to $\sim 5\%$ at production gaps (5.7/4.1/5.4% at 0.05/0.1/0.2 mm). This validates the *solver* against a measured datasheet, not the GNN, which the EER40 case does not feed. Placing our layouts on that core is what puts the magnetizing inductance 37-73 $\times$ above the air-core M .

Frequency-dependent extraction. AC winding resistance rises with frequency through skin and proximity effects, so we extend the surrogate from scalars to the primary $R_{ac}/R_{dc}(f)$ curve over 10 kHz-100 MHz (17 points), ground-truthed by a FASTHENRY sweep with

Table 4. All four parasitics field-grade: the trained GNN vs. 3-D field solvers (FASTHENRY for L , an electrostatic FEM for C_{ps}), and the analytical labels they replace.

Analytical Grover $L_p/L_s/M$ vs. FASTHENRY 3-D	52/54/59 %
Analytical C_{ps} vs. FEM-3D (position-blind overlap)	267 %
GNN $L_p/L_s/M$ vs. FASTHENRY 3-D	2.8/3.8/2.6 %
GNN C_{ps} vs. 3-D electrostatic FEM ($R^2=0.95$)	3.2 %
Real core (EER40 GP95) FEM vs. datasheet A_L , prod. gaps	$\sim 5\%$
Large- N measured, test 51-70 nodes, min (R^2)	≥ 0.988

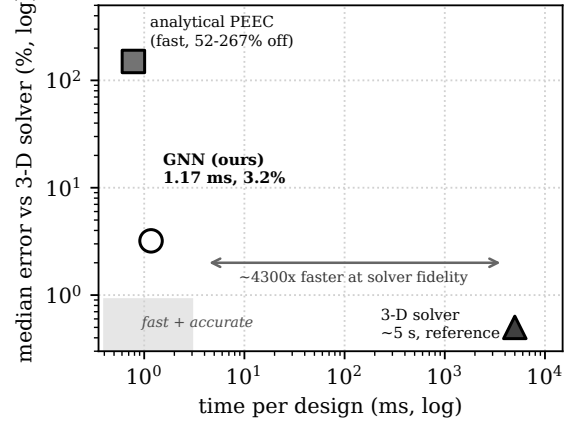


Figure 2. Speed-accuracy trade-off (both axes log). The analytical PEEC is fast but 52-267% off the 3-D solvers; the solvers are accurate but ~ 5 s/design; the GNN occupies the fast-and-accurate corner (3.2% median at 1.17 ms).

filament subdivision (mean curve $1.0 \rightarrow 6.0 \times$). Trained on 136 small boards, the same GNN with a 17-dimensional head predicts held-out curves to **0.37% median** error (curve $R^2 = 0.995$), closing the “low-frequency only” gap.

Does equivariance help? It does not. An $E(n)$ -equivariant network (EGNN [18], pure PyTorch) is *marginally worse* on all four targets (R^2 eq/MPNN: 0.9993/0.9997, 0.9986/0.9992, 0.9983/0.9995, 0.9996/0.9998), with no advantage in a sample-efficiency sweep (200/800 graphs); its only benefit is near rotation-invariance (0.33% median drift under a random 3-D rotation). Because the targets are aggregated invariant scalars, the MPNN’s relative-geometry edge features already encode the needed symmetry.

D. Speed at solver-grade accuracy

The design-loop value is a *speed-accuracy* one (Fig. 2). There are three ways to get the parasitic vector for a candidate stack-up: the analytical PEEC (0.78 ms/design, but 52-267% off the 3-D solvers); the 3-D solvers FASTHENRY+FEM (~ 5 s/design, the accurate reference); or the GNN, which returns the same four parasitics at *field-grade* accuracy in **1.17 ms/design, $\sim 4,300\times$ faster than the solvers it matches**. It is thus the only option that is *both* fast and 3-D-accurate: the fast analytical model is the wrong number, and the right number is three orders of magnitude too slow for an inner loop. Screening 1000 interleavings costs ~ 1.2 s with the GNN versus ~ 1.4 h with the solvers, after which only the top few candidates need a confirming 3-D solve.

Versus the fast baseline. Even against the (fast but inaccurate) analytical PEEC the GNN is competitive at scale: a dense all-pairs graph is $O(N^2)$ (1.17 vs. 0.78 ms at the 25-node mean, the GNN slower), but a sparse k -NN graph ($k = 8$, retrained, $R^2 \geq 0.992$) is $\approx O(N)$ and overtakes the unaccelerated PEEC at $N \approx 80$ segments, $\sim 80\times$ at $N = 1280$ (extrapolated accuracy there); a multipole PEEC would itself approach $O(N)$. The headline claim remains the 4,300 $\times$ over the *accurate* solvers, which no fast analytical model matches for accuracy.

V. DISCUSSION AND LIMITATIONS

The accuracy is reproducible across seeds, the graph structure is shown to be necessary, all four targets are validated against 3-D field solvers,

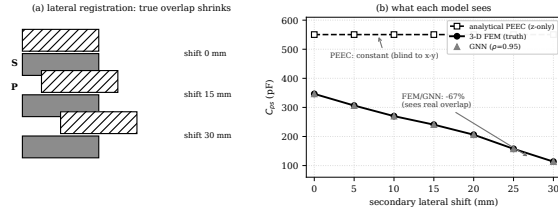


Figure 3. Why the GNN’s ranking beats the analytical model on lateral registration. (a) Shifting the secondary trace laterally shrinks the true P/S overlap. (b) The analytical C_{ps} uses $\min(L) \min(w)$ and is position-blind, so it stays constant (550 pF) and cannot rank; the 3-D FEM (and the GNN tracking it at $\rho = 0.95$) follows the real overlap and drops 67%.

the physics is checked against a real commercial core, and the fast sparse model is itself accuracy-validated. Four limitations remain. (i) Validation is solver-grade, not hardware-grade: “field-grade” means matching a solver, not a measured board. The cross-solver check (Sec. C) rules out single-solver overfitting but not a shared modelling assumption, the scope is the winding terms rather than the core magnetizing inductance, and the C_{ps} FEM uses a uniform effective permittivity. Full-wave/Q3D and silicon anchoring remain open. (ii) The geometry is synthetic (random interleaved rectangular foils; no vias, return planes, or board-level parasitics). (iii) An $E(n)$ -equivariant architecture did *not* improve accuracy or sample efficiency; the gains here come from better labels and a frequency target, not architectural symmetry.

(iv) **Ranking: a ranking objective, and the lever where it matters.** First, ranking needs a ranking objective: a pointwise regressor orders within-family variants at only $\rho = 0.27$ (family-disjoint), because at global $R^2 = 0.95$ its error floor ($\sqrt{1 - R^2} \cdot \sigma$) rivals the small within-family C_{ps} spread, so a globally-excellent regressor is not a fine-grained ranker (an earlier $\rho = 0.51$ -0.74 was a leakage artifact). A pairwise margin-ranking objective on the FEM-3D labels, in the learning-to-rank tradition [28, 29], fixes this ($\rho = 0.93$). Second, where it beats the free analytical baseline: an optimiser turns two unequal levers. For **interleaving** (P/S layer/ z order) the analytical PEEC is already competitive ($\rho = 0.78$ vs. the FEM; GNN 0.85), because the z -gap is in its parallel-plate formula. For **lateral registration** (in-plane x - y overlap, Fig. 3) the PEEC is *blind*: its overlap $\min(L) \min(w)$ is position-independent, so it returns a *constant* C_{ps} across every test family and cannot rank at all, while the FEM C_{ps} varies by $\sim 100\%$ and the GNN tracks it at $\rho = 0.95$. This changes the design choice: picking the lowest- C_{ps} layout per family, the PEEC-screened pick carries 1176% regret vs. the FEM optimum—no better than random (1126%), the screener being blind—whereas the GNN-screened pick has 3.5%, a $\sim 340\times$ better decision. On interleaving both pick well (5.4 vs. 5.8%). The residual cap is that these decisions are against the FEM, not a measured board.

Integration. In the inner loop of a layout optimiser the regression head returns absolute $[C_{ps}, L_p, L_s, M]$ in ~ 1 ms and the pairwise head orders unseen designs at $\rho = 0.93$, so hundreds of candidates are screened before any field solve; the ~ 1 MB pure-PyTorch checkpoint ships inside the open tool and runs on a laptop CPU.

VI. CONCLUSION

A license-clean geometry-aware GNN predicts the planar-PCB C_{ps} and air-core inductance vector at $R^2 > 0.999$ over three seeds, with accuracy shown to require message passing and geometry. After diagnosing a 62.6% Grover-label error and regenerating every label with its 3-D solver, all four targets are field-grade (2.6-3.8% on $L_p/L_s/M$, 3.2% on C_{ps}), an independent Neumann reference places the GNN inside the inter-solver gap, and the separately-validated core term is quantified at 37 - $73\times$ the winding mutual. We extend the surrogate to $R_{ac}/R_{dc}(f)$ at 0.37% and fix the ranking gap ($\rho = 0.27 \rightarrow 0.93$, decisive on the lateral lever where the analytical model is blind), and re-

port two honest negatives ($E(n)$ -equivariance gives no gain; the dense graph is slower than analytical, the sparse k -NN variant overtaking it at $N \approx 80$ at $R^2 \geq 0.992$). Next: silicon/Q3D anchoring, vias and return planes, and an operator-learning surrogate [26, 27] for the full field.

REPRODUCIBILITY

The release includes the dataset generator (fixed seed), per-run logs for every experiment reported here, the trained checkpoints, and the FEM and FASTHENRY reference solvers (built from source, MIT-licensed). The code (pure PyTorch/NumPy/SciPy/scikit-fem) is BSD-3; every number regenerates from these artifacts.

REFERENCES

- [1] Z. Ouyang and M. A. E. Andersen, “Overview of planar magnetic technology—Fundamental properties,” *IEEE Trans. Power Electron.*, vol. 29, no. 9, pp. 4888–4900, 2014.
- [2] W. G. Hurley and W. H. Wölflé, *Transformers and Inductors for Power Electronics*. Wiley, 2013.
- [3] C. W. T. McLyman, *Transformer and Inductor Design Handbook*, 4th ed. CRC Press, 2011.
- [4] R. W. Erickson and D. Maksimović, *Fundamentals of Power Electronics*, 3rd ed. Springer, 2020.
- [5] P. L. Dowell, “Effects of eddy currents in transformer windings,” *Proc. IEE*, vol. 113, no. 8, pp. 1387–1394, 1966.
- [6] W. A. Roshen, “Fringing field formulas and winding loss due to an air gap,” *IEEE Trans. Magn.*, vol. 43, no. 8, pp. 3387–3394, 2007.
- [7] A. E. Ruehli, “Equivalent circuit models for three-dimensional multiconductor systems,” *IEEE Trans. Microw. Theory Tech.*, vol. 22, no. 3, pp. 216–221, 1974.
- [8] M. Kamon, M. J. Tsuk, and J. K. White, “FASTHENRY: A multipole-accelerated 3-D inductance extraction program,” *IEEE Trans. Microw. Theory Tech.*, vol. 42, no. 9, pp. 1750–1758, 1994.
- [9] F. W. Grover, *Inductance Calculations: Working Formulas and Tables*. Van Nostrand, 1946 (repr. Dover, 1962).
- [10] H. W. Ott, *Electromagnetic Compatibility Engineering*. Wiley, 2009.
- [11] C. R. Paul, *Introduction to Electromagnetic Compatibility*, 2nd ed. Wiley, 2006.
- [12] T. Gustafsson and G. D. McBain, “scikit-fem: A Python package for finite element assembly,” *J. Open Source Softw.*, vol. 5, no. 52, 2369, 2020.
- [13] C. Geuzaine and J.-F. Remacle, “Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities,” *Int. J. Numer. Methods Eng.*, vol. 79, no. 11, pp. 1309–1331, 2009.
- [14] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *NeurIPS*, 2019.
- [15] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *ICML*, 2017, pp. 1263–1272.
- [16] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *ICLR*, 2017.
- [17] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *ICLR*, 2019.
- [18] V. G. Satorras, E. Hoogeboom, and M. Welling, “E(n) equivariant graph neural networks,” in *ICML*, 2021.
- [19] A. Sanchez-Gonzalez *et al.*, “Learning to simulate complex physics with graph networks,” in *ICML*, 2020, pp. 8459–8468.
- [20] G. Huang *et al.*, “Machine learning for electronic design automation: A survey,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 26, no. 5, Art. 40, pp. 40:1–40:46, 2021.
- [21] H. Ren *et al.*, “ParaGraph: Layout parasitics and device parameter prediction using graph neural networks,” in *DAC*, 2020.
- [22] G. Zhang, H. He, C.-Y. Hsu, and D. Katabi, “Circuit-GNN: Graph neural networks for distributed circuit design,” in *ICML*, 2019, pp. 7364–7373.
- [23] A. Mirhoseini *et al.*, “A graph placement methodology for fast chip design,” *Nature*, vol. 594, pp. 207–212, 2021.
- [24] Z. Xie *et al.*, “Net2: A graph attention network method customized for pre-placement net length estimation,” in *ASP-DAC*, 2021.
- [25] K. Nabors and J. White, “FastCap: A multipole accelerated 3-D capacitance extraction program,” *IEEE Trans. Comput.-Aided Design*, vol. 10, no. 11, pp. 1447–1459, 1991.
- [26] Z. Li *et al.*, “Fourier neural operator for parametric partial differential equations,” in *ICLR*, 2021.
- [27] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators,” *Nat. Mach. Intell.*, vol. 3, pp. 218–229, 2021.
- [28] C. J. C. Burges *et al.*, “Learning to rank using gradient descent,” in *ICML*, 2005.
- [29] C. J. C. Burges, “From RankNet to LambdaRank to LambdaMART: An overview,” Microsoft Research, Tech. Rep. MSR-TR-2010-82, 2010.
- [30] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *ICLR*, 2019.